2009 Sixth IFIP International Conference on Network and Parallel Computing

GFFC: the Global Feedback based Flow Control in the NoC Design for Many-core Processor

Xu Wang †Ge Gan ‡Dongrui Fan Shuxu Guo

Department of EE
Jilin University
Jilin, China 130000
{wangxu,guosx}@jlu.edu.cn

†Department of ECE
University of Delaware
Newark, DE. 19716
gan@capsl.udel.edu

‡Key Lab of Computer System and Arch
ICT, Chinese Academy of Sciences
Beijing, China 100190
fandr@ict.ac.cn

Abstract

GFFC (Global Feedback based Flow Control) is proposed to be used in NoC design for many-core processor. GFFC is designed based on two fundamental principles: (a) when network congestion occurs, the packet sender that causes the congestion needs to know this and needs to be proactively involved in the alleviation of this network congestion; (b) the congestion-causing packets shall not block the traffic of other part of the OCN. Following these principles, we propose the GFFC FIFO in the OCN router. GFFC FIFO is a place to buffer the waiting packets in the OCN to avoid blocking the traffic of other bypassing packets. Besides, we propose a mechanism to relay the congestion information from the packet receiver to the packet sender via the GFFC FIFO in the routers. In this paper, we propose the design of GFFC in a mesh based OCN. We also study the performance characteristics of GFFC and report the experimental results. The experimental results show that GFFC can decrease the average non-memory packet transfer latency by 9%.

1 Introduction

With the development of computer architecture and CMOS technology, more and more processor cores are able to be integrated on a single chip. In today's market, processors with 2-10 cores are very common. At the same time, chip companies like Intel and nVidia are planning to ship processors with about 10-100 cores in the near future [1] [2]. It is even anticipated that, by the end of the next decade, the number of on-chip cores will reach 1000 [3] [4] [5]. However, how to connect so many on-chip processor cores (and other on-chip resources like L1/L2 cache and memory & I/O controllers) and provide them with high performance communication (high bandwidth and low latency) remains a very challenging problem.

To solve this problem, many on-chip interconnection designs were proposed, developed, and experimented. Among all of these NoC designs, the mesh-based on-chip network (OCN) with an X-Y routing algorithm Such a design was adopted in several famous many-core processor projects due to its simplicity, flexibility, and scalability. In addition, people also proposed and studied a variety of flow control mechanisms for the mesh-based OCN.

This paper will introduce a new flow control mechanism which is called Global Feedback based Flow Control, or GFFC for short. GFFC allocates network resources (i.e. buffer space and channel bandwidth) based on the feedback signals from the OCN nodes that causes network congestion. Routers around the congestion OCN node will be notified of the traffic jam so the bypassing OCN packets destined to the congestion node will be pulled over and placed in a temporary buffer. Therefore, other bypassing packets destined to other places will not be blocked by unrelated traffic jam. This is different from the previous methods in which only local information is considered in deciding how router buffer and channel bandwidth are allocated to bypassing packets [6] [7]. In GFFC, the global traffic congestion information is taken into account in network flow control. This is like on the highway where vehicles destined to the jammed exit can be pulled over so the highway will not be blocked.

The design of the new flow control mechanism is based on the observation that, in the on-chip network used by a many-core processor, most of the traffic congestions are caused by burst requests to a few global shared on-chip resources. For instance, the on-chip L2 cache banks, memory controllers, and I/O controllers [8]. To make the paper easy to follow, we call these global shared resources as *congestion hot-spots* in OCN. If there are burst memory requests,

978-0-7695-3837-2/09 $26.00 © 2009 IEEE
DOI 10.1109/NPC.2009.16

IEEE computer society

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on August 24,2026 at 09:33:05 UTC from IEEE Xplore. Restrictions apply.

the request packets will pile up in the queues of the congestion hot-spots. The buffer space in the congestion hot-spots will be exhausted and the continuing arrival packets will eat up the buffer space in the neighboring routers and finally block other bypassing packets destined to elsewhere. Due to the long latency of memory request handling, it is not possible to eliminate this kind of traffic jams on the sink side. The correct fix relies on slowing down the speed of the source side, i.e. making the burst requests more graceful. GFFC provides the channel to feedback the congestion information to the source node so the source node can slow down issuing outing packets.

The major contributions of this paper are: **(a)** we propose GFFC, a new flow control mechanism that allocates OCN resources based on the feedback from the *congestion hot-spots* in the OCN. As far as the authors are aware, this is the first paper that study this kind of flow control mechanism on OCN designed for many-core processor. **(b)** we introduce our implementation of GFFC on a many-core processor simulator. We present the details how GFFC works in a 2D-mesh OCN and how to design the OCN router to support GFFC. **(c)** we report the evaluation results of GFFC. The experimental results demonstrate that, with the GFFC flow control, the average non-memory packet transfer latency has decreased 9%.

The rest of the paper is organized as follows. Section 2 gives a quick introduction on the background of our new flow control mechanism and uses a motivating example to demonstrate why feedback information is important in resource allocation. Section 3 introduces what is global feedback flow control and how it can be implemented. We present our experimental data in section 4 and provide our conclusions in section 5.

2 Motivation

In this section, we give an example to show the problem caused by burst requests in OCN. Then we discuss how the feedback information can help to solve the problem.

Figure 1(a) shows part of the OCN that connects to a congestion hot-spot. We assume that the OCN adopts a 2D mesh topology. Each node in the mesh is a router with five inbound and five outbound ports (*east*, *west*, *north*, *south*, and *local*, to make the figure easy to read, we omit the *local* port in Figure 1). Each port is depicted as a buffer queue with the corresponding arrow to specify its direction.

In Figure 1(a), we assume that there are burst requests to access the shared critical resource (which is denoted as *congestion hot-spot*) on the right hand side The request packets flood into *Router2* from the north direction and are routed to *Router3*, which would forward the packets to their destination. If the request packets can not be processed enough quickly by the receiver, they would pile up in the corresponding buffer queues in *Router3*. Soon, the jam will be propagated backward to *Router2* and also other routers on the route from the source to the destination. Since these packets can not be discarded before they arrive their destination, they will stay in the buffer queues of the corresponding OCN routers for maybe a very long time. This would block the traffic of other bypassing packets. For instance, in Figure 1, no packet can be fed into *Router2* from its north inbound port, even the packet is supposed to be forwarded to elsewhere. The same would happen in *Router3* and all OCN routers jammed by these packets.

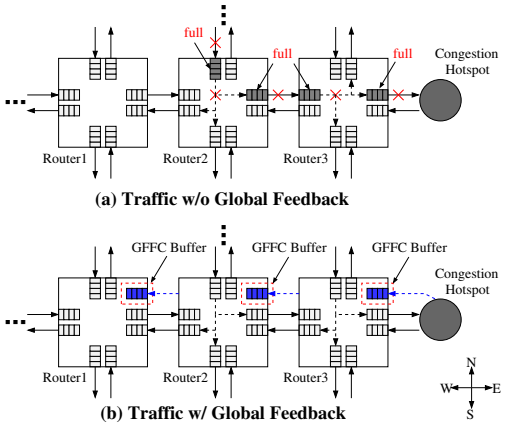


Figure 1. Flow Control w/o Global Feedback and Flow Control w/ Global Feedback

This kind of congestion can only be eliminated or alleviated if we can: **(a)** speedup the packet processing on the sink side; or **(b)** slowdown the packet sending on the source side. **(a)** is not a problem in the realm of OCN design. For **(b)**, none of the current flow control methods [6] [7] provide a mechanism to feedback the congestion information from sink to source. This motivates us to design a new flow control - the global feedback based flow control (GFFC for short). Figure 1(b) shows the basic idea of GFFC. Each shared critical resource (e.g. on-chip L2 cache bank) connected into the OCN has a dedicated buffer allocated for it in each OCN router. We call the shared critical resources as *congestion hot-spots* because they cause most of the congestions in the network. We call the dedicated router buffer as its *GFFC buffer*.

At the normal time, GFFC buffer can be used as the storage for virtual channel [6]. In case congestion happens in a *congestion hot-spot* in OCN, the associated GFFC buffer will be enabled by a signal from the corresponding *congestion hot-spot*. In an OCN router with an enabled GFFC buffer, any packet destined to the associated *congestion hot-spot* will not be forwarded to its next hop. Instead, the packet will be stored in the GFFC buffer until the buffer gets a disable signal from the associated congestion hot-

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on August 24,2026 at 09:33:05 UTC from IEEE Xplore. Restrictions apply.

spot. The GFFC buffer enabling signal is propagated from the congestion hot-spot to the source node in a wavefront pattern. An enabling signal is forwarded (or backward-ed) to the neighboring router only if the GFFC buffer in the current router is full. Follow this pattern, the congestion information is transferred from the congestion hot-spot to the source node. So the source node can stop pouring huge amount of packets into the OCN. Besides, the waiting packets are stored in the dedicated GFFC buffer. They do not block the inbound/outbound ports of the router. So other bypassing packets can still travel to their destination via this router.

3 Global Feedback based Flow Control

In this section, we first present an overview of the design of GFFC, including the GFFC mesh OCN and the GFFC router. Then, we explain how GFFC router works using the GFFC finite state automata. At the end of this section, we introduce three different GFFC enable policies and compare their similarities/differences.

3.1 GFFC: Network Architecture and Router Design

Figure 2 shows the design of GFFC in a typical 2D mesh-based OCN. The diagram on the left hand side is the scheme of a many-core processor that connected by a 5×5 2D mesh-based OCN. Each node in the mesh network is a GFFC router, which is illustrated in Figure 2(b). Every on-chip functional unit like processor core, cache bank, and memory controller is attached to one mesh router. They communicate with each other by sending packets to the local router. The local router, together with other routers in the OCN, use the packet-switching [9] technique to forward the packets to the specified destination based on the predefined routing algorithm, e.g. the X-Y routing algorithm [10]. To make the diagrams easy to read, all unrelated on-chip functional units are omitted in Figure 2.

Basically, a GFFC router is similar to a traditional mesh-based OCN router used in [6]. Its major components are inports/outports (including buffers), router computation logic, switch allocator, crossbar switch, or virtual channel allocator, etc. Each mesh router has five inports and five outports (*north*, *south*, *east*, *west*, and *local*). The processor core is usually connected to the router's *local* port. The arbitrators decide the next packet to be sent out. After the arbitrators select the candidate packet for sending (usually the first packet from one of the inport buffers that wins the arbitration), the packet will be directly put in the outport buffer (the last empty slot in the buffer), or the inport buffer of the next hop.

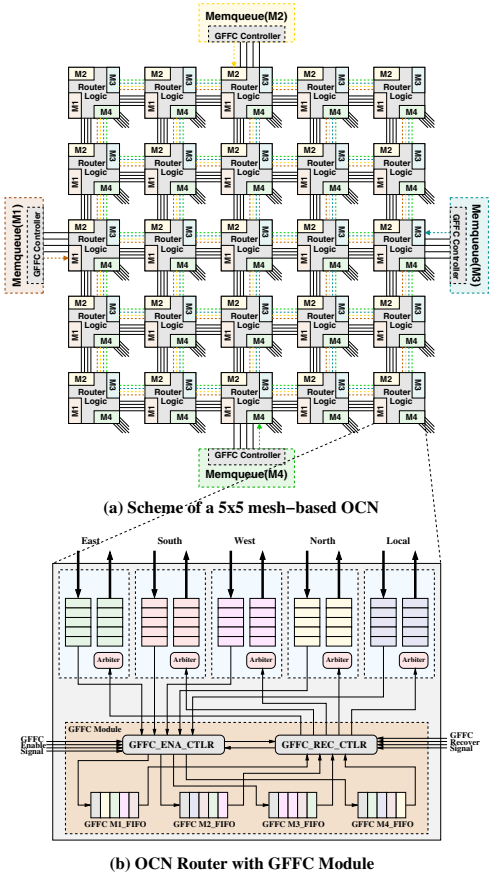


Figure 2. GFFC System Architecture

Besides the traditional functional blocks, the GFFC mesh router has an additional *GFFC module*. The major components in the GFFC module are the *GFFC enable controller* (denoted as `GFFC_ENA_CTLR`), the *GFFC recover controller* (denoted as `GFFC_REC_CTLR`), and the *GFFC FIFO*s. `GFFC_ENA_CTLR` controls the data path from the inport buffers to the GFFC FIFO and `GFFC_REC_CTLR` controls the data path from GFFC FIFO to the outport buffers.

As we mentioned in the last section, every congestion hot-spot of the OCN is given a dedicated GFFC FIFO in every GFFC router. In Figure 2(a), the memory queues in the memory controllers are identified as congestion hot-spots. Therefore, each of them has a GFFC FIFO in the router. When the memory queue in a specific memory controller is full, it will send out a signal via its local GFFC controller to enable the GFFC module in the nearest GFFC router (i.e. the local GFFC router). This signal will wake up the router's GFFC enable controller, i.e. `GFFC_ENA_CTLR`. After the `GFFC_ENA_CTLR` is turned on, packets destined to that congestion hot-spot will not be forwarded to the outport calculated by the routing algorithm. Instead, it will be

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on August 24,2026 at 09:33:05 UTC from IEEE Xplore. Restrictions apply.

forwarded to the corresponding GFFC FIFO of the router and stored there.

During burst access, a great number of packets are sent to one congestion hot-spot in a very short amount of time. Because of the high rate of incoming packets, the space in the GFFC FIFO will soon be exhausted. Under this condition, the current router will send the GFFC enable signal to the surrounding routers which, after receiving the signal, would buffer all congestion-causing packets in the corresponding GFFC FIFOs. Via this wavefront pattern, the GFFC enable signal will be gradually propagated to the source node that sends out the congestion-causing packets. This signal is a very useful feedback from the receiving side. It carries the information that tells the sender that the sending rate is too high for the receiver to handle all incoming packets in time, therefore causing significant network congestions. The sender, upon receiving this signal, can choose to stop its execution pipeline or suspend issuing related OCN packets to the receiver. This prevents more congestion-causing packets from being poured into the OCN, which would greatly reduce the number of waiting packets in the OCN. Besides, the congestion-causing packets that have already been sent into the OCN are buffered in the dedicated GFFC FIFOs. So, the inport and outport buffer space is saved and can be used by other bypassing packets. This avoids further network congestion and can greatly increase the OCN resource utilization.

With time passing by, the waiting packets in the queue of the congestion hot-spot (i.e. memory queues in our case) will be gradually drained out. This means that the network congestion starts to disappear. Therefore, sooner or later, there will be available space in the queue for incoming packets. Under this condition, the GFFC controller in the congestion hot-spot would send out a *GFFC recover signal* to the local router. This signal will turn on the GFFC recover controller, i.e. `GFFC_REC_CTLR` in the router. `GFFC_REC_CTLR` will open the data path from the corresponding GFFC FIFO to the outport. This allows the packets buffered in the GFFC FIFO start to flow to their destination. After the number of buffered packets in the GFFC FIFO reduces to a predefined threshold, `GFFC_REC_CTLR` will propagate another *GFFC recover signal* to the surrounding routers of the next stage. At the same time, `GFFC_REC_CTLR` sends a turn-off signal to `GFFC_ENA_CTLR` so it would close the data path from the inport buffers to the GFFC FIFO. This means that the succeeding incoming packets destined to the original congestion hot-spot would flow directly from the router's inports to the router's outport. With the *GFFC recover signal* being gradually propagated back to the source node, the network path from the sender to the receiver gets unstuffed. Therefore, the packets buffered in the GFFC FIFOs along the data path start to flow to their destination. The *GFFC recover signal* will be finally sent to the sender and the sender will resume its normal execution.

3.2 GFFC Finite State Automata

Figure 3 shows the scheme of the GFFC finite state automata. We can use six states to depict the runtime behaviors of the GFFC router. They are IDLE, CHECK, KEEP, BROADCAST ENABLE, RECOVER, and BROADCAST RECOVER.

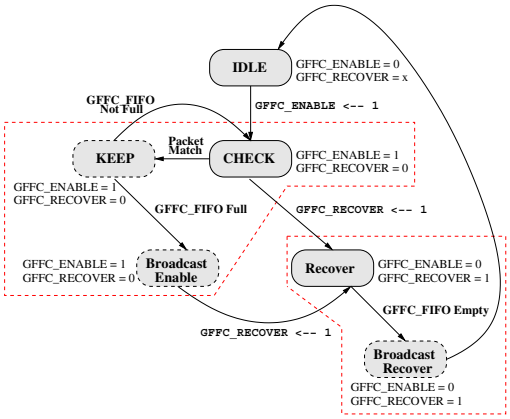


Figure 3. GFFC Finite State Automata

At the very beginning, there is no traffic jam in the pre-identified congestion hot-spot. So the GFFC router is in IDLE state. In this state, the GFFC module in the router is disabled and the value of the register `GFFC_ENABLE` is zero. This closes the data paths from the router's inports to the corresponding GFFC FIFO, therefore, all incoming packets are directly forwarded to the router's outports that are calculated by the pre-defined routing algorithm.

When traffic jam happens in a congestion hot-spot, a GFFC enable signal will be sent out from this congestion hot-spot to the routers that forward/send packets to it. This signal turns on the routers' GFFC modules and transfers the routers from IDLE state to CHECK state. It also sets the register `GFFC_ENABLE` to 1 and sets the register `GFFC_RECOVER` to 0. In CHECK state, `GFFC_ENA_CTLR` starts to intercept the packets destined to the corresponding congestion hot-spot and sent them to the GFFC FIFO associated with that congestion hot-spot. Two sub-states, KEEP and BROADCAST ENABLE, are involved in this procedure. If `GFFC_ENA_CTLR` gets a congestion-causing packet, the router will enter KEEP state. In this state, the packet will be sent to the related GFFC FIFO and be buffered there. If the FIFO is not full, the router returns to CHECK state. Otherwise, the router enters BROADCAST ENABLE, in which, it relays the GFFC enable signal to the routers at the next stage.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on August 24,2026 at 09:33:05 UTC from IEEE Xplore. Restrictions apply.

After the network traffic jam disappears in the congestion hot-spot, it will propagate the GFFC recover signal to the network. Upon receiving the recover signal, the GFFC router is transferred from CHECK or BROADCAST ENABLE to RECOVER state. In RECOVER state, the GFFC router set register `GFFC_REC_CTLR` to 1. This closes the data paths from the router's inports to the related GFFC FIFO. In addition, it opens the data paths from GFFC FIFO to the router's outports so the buffered packets in the GFFC FIFO start to flow their destination. After the GFFC FIFO becomes empty, the router will enter BROADCAST RECOVER state. In this state, the router will relay the GFFC recover signal to the surrounding routers. After that, the GFFC module is closed and the router enter GFFC IDLE state.

4 Experimental Results

We have developed a cycle accurate OCN simulator to evaluate the performance of GFFC. Figure 4 shows the parameters of this OCN simulator.

OCN topology	8×8 mesh
Hop latency	3 cycles
Inport Queue Capacity	5 Entries
Arbiter FIFO Capacity	4 Entries
# Congestion hot-spot	4
Memory Access Latency	50 cycles
Memory Queue Capacity	15 Entries
GFFC FIFO Capacity	5 Entries
GFFC FIFO In	1 cycle
GFFC FIFO Out	1 cycle

Figure 4. OCN Simulator Parameters

The simulator simulates a 8×8 2D mesh network. Each router takes 3 cycles (routing/arbitration/deliver) to forward a packet[1] to the next hop. Each inport of the router has a 5-entry queue to buffer incoming packets. Each arbiter of the router can hold 4 packets in its FIFO. The simulator has simulated 4 congestion hot-spots, which are 4 memory controllers on the chip. The queue in each memory controller can hold 15 memory request packets. It takes 50 cycles to finish a memory access, i.e. each packet need to stay in the memory queue for at least 50 cycles. The router has 4 GFFC FIFOs, which correspond to the 4 congestion hot-spots. Each GFFC FIFO can keep 5 packets. It takes each packet 1 cycle to enter and exit the GFFC FIFO. To evaluate the performance impact of GFFC on the OCN traffic, we have injected $100,000$ packets into the simulator and measured the average packet transfer latency with GFFC turned on and with GFFC turned off.

Two different kinds of packets were injected into the simulator. One is called *memory packets*, which are destined to the four memory controllers, i.e. the congestion hot-spots. The other kind is called *non-memory packets*, which are destined to elsewhere. Figure 5 shows the percentage of two types of packet latency decrement under different memory packet ratios. GFFC can not help to reduce the memory access latency because the memory access latency is mainly decided by the latency of the interaction between the on-chip memory controller and the off-chip memory module. However, GFFC can optimize the transfer latency of the non-memory packets. The grey bars in Figure 5 represent the average percentage of latency improvement of non-memory packets. The dark bars in Figure 5 represent the average percentage of latency improvement of all kinds of packets. All the grey bars are higher than the dark bars because the latencies of the memory packets are not affected by GFFC. Besides, the greatest improvement is obtained when the percentage of memory packets in all packets reaches 60%.

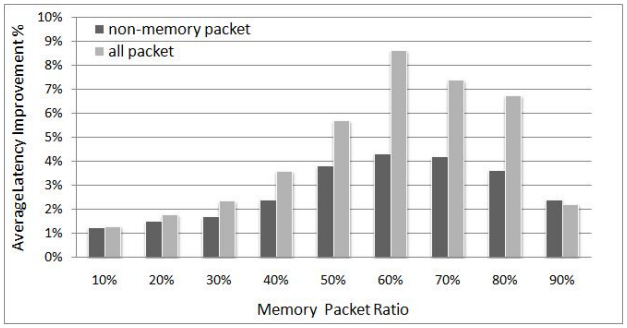


Figure 5. Latency Improvement Under Different Ratio of Memory Packets

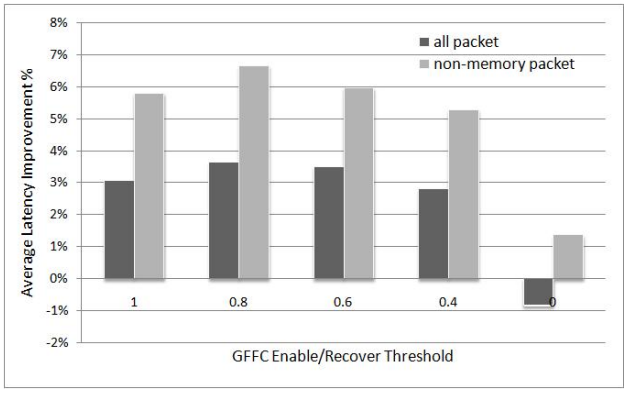


Figure 6. Latency Improvement Under Different GFFC Enable/Recover Thresholds

Figure 6 shows the percentage of the average packet latency improvement under different GFFC enable/recover threshold. A very interesting observation is that, when the

[1] In this paper, we assume one packet is one flit.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on August 24,2026 at 09:33:05 UTC from IEEE Xplore. Restrictions apply.

GFFC enable/recover threshold was set to 0, i.e. zero tolerant or the *eager* enable/recover policy, there are performance degradation on the overall packet transfer latency. The reason is that, with the *eager* enable/recover policy, it only takes the OCN at most 8 cycles to propagate the GFFC enable/recover signal across all OCN routers. Besides, each GFFC FIFO can only buffer one packet. This results in that, most of the packets destined to the memory controller will be buffered in the GFFC FIFO of the routers that it would come across on its road to destination. Therefore, every such kind of packet will have to stay in each router on its path for at least two more cycles (GFFC FIFO in and out). This is where the performance degradation comes from. Actually, as we said, not all packets destined to congestion hot-spot must trigger GFFC. GFFC is only supposed to be triggered by the huge number of burst request to a few congestion hot-spots. The GFFC *eager* enable/recover policy makes the GFFC module too sensitive, which is the root cause of performance degradation.

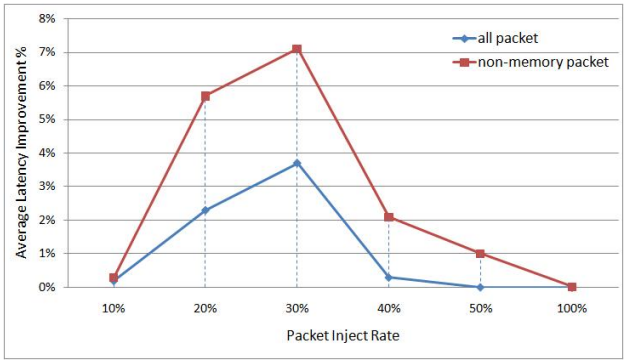


Figure 7. Latency Improvement Under Different Packet Injection Rate

Figure 7 shows the improvement of the packet transfer latency under different packet injection rate. For each measurement, we have injected $100,000$ packets into the OCN. These packets can be injected into the OCN in different rate. For example, for the injection rate 25%, at every cycle, one packet is injected into each of the one fourth of the OCN routers. The one fourth of the OCN routers are randomly selected at every cycle. From Figure 7, we can see that the best performance improvement was obtained when the packet injection rate was set to 30%. The performance improvement brought by GFFC is too small and can be ignored when the injection rate is too high (close to 100%) or too low (10%). The explanation is that, when the packet injection rate is too low, there are not many network congestions. So, there are not many opportunities for the GFFC to be turned on. On the other hand, when the packet injection rate is too high, the OCN is saturated by the injected packets. So, there is no difference whether GFFC is turned on or off.

References

[1] Held, J., Bautista, J., Koehl, S.: From a few cores to many: A tera-scale computing research overvi ew. Intel White Paper (2006)

[2] Lindholm, E., Nickolls, J., Oberman, S., Montrym, J.: Nvidia tesla: A unified graphics and computing architecture. IEEE Micro **28**(2) (2008) 39–55

[3] Borkar, S.: Thousand core chips: a technology perspective. In: DAC '07: Proceedings of the 44th annual conference on Design automation, New York, NY, USA, ACM (2007) 746–749

[4] Asanovic, K., Bodik, R., an d Joseph James Gebis, B.C.C., Husbands, P., Keutzer, K., rson, D.A.P., Plishker, W.L., Shalf, J., an d Katherine A. Yelick, S.W.W.: The landscape of parallel computing research: A view from berkel ey. Technical Report UCB/EECS-2006-183, EECS Department, University of California, Berkeley (December 2006)

[5] Yeh, D., Peh, L.S., Borkar, S., Darringer, J., Agarwal, A., mei Hwu, W.: Thousand-core chips. IEEE Design and Test of Computers **25**(3) (2008) 272–278

[6] Dally, W.J.: Virtual-channel flow control. IEEE Trans. Parallel Distrib. Syst. **3**(2) (1992) 194–205

[7] Peh, L.S., Dally, W.J.: Flit-reservation flow control. In: Proceedings of the Sixth International Symposium on High-Performance Computer Architecture, Toulouse, France, IEEE Computer Society (January 8–12, 2000) 73–84

[8] Wang, X., Gan, G., Manzano, J., Fan, D., Guo, S.: A quantitative study of the on-chip network and memory hierarchy design for many-core processor. International Conference on Parallel and Distributed Systems **0** (2008) 689–696

[9] Cyper, R.J.: Communications Architecture for Distributed Systems. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA (1978)

[10] Ni, L.M., McKinley, P.K.: A survey of wormhole routing techniques in direct networks. Computer **26**(2) (1993) 62–76

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on August 24,2026 at 09:33:05 UTC from IEEE Xplore. Restrictions apply.